PROJECT REPORT

Cloud-Based Linux Server Deployment with GitHub Version Control and Automation

Student Name	Vaibhav Rane
Batch Code	A530
Project Repository	https://github.com/VaibhavR12-Gh/DevOps-Fundamentals-Project
Platform	AWS EC2 Ubuntu, Apache, Git and GitHub
Report Date	24 October 2026

Prepared from the supplied ITVedant assignment brief and the accessible README in the linked GitHub repository. Implementation details that could not be independently verified are identified in the report.

Abstract

This mini-project demonstrates a basic cloud-hosted website deployment workflow using an Ubuntu server on Amazon Web Services (AWS), the Apache web server, and Git/GitHub for version control. It is intended to build practical familiarity with cloud networking, Linux package management, website hosting, shell automation, and technical documentation.

1. Introduction

Cloud infrastructure enables applications and websites to be hosted on remotely managed compute resources. In this project, an Ubuntu EC2 instance is used as the Linux host for a simple static webpage. Apache serves the webpage, while Git and GitHub provide a way to track project files and publish the project repository.

The submitted assignment brief also includes server health monitoring, Apache service recovery, website backups, and log analysis as expected project tasks. These tasks are described in this report as the intended operational workflow; successful execution should be supported by the actual scripts, generated report, and screenshots in the repository.

2. Project Objectives

• Configure AWS networking resources, including a custom VPC and public subnet.

• Launch and access an Ubuntu EC2 instance using SSH.

• Update Ubuntu and install required tools such as Git, Apache, Curl, and Vim.

• Deploy a static HTML webpage and verify it through the EC2 public IP address.

• Use Git for staging, commits, branching, merging, and pushing to GitHub.

• Document the intended Bash automation and log-monitoring activities.

3. Scope and Technologies

Cloud platform: AWS. Compute: Amazon EC2 running Ubuntu. Networking: VPC, public subnet, Internet Gateway, route table, and security group. Web server: Apache HTTP Server. Version control: Git and GitHub. Scripting and administration: Bash and standard Linux utilities. Web content: HTML.

4. AWS Architecture

The architecture described in the project README is: VPC (10.0.0.0/16) → Public Subnet (10.0.1.0/24) → Internet Gateway → Route Table with a default route (0.0.0.0/0) → Security Group allowing SSH (TCP 22) and HTTP (TCP 80) → Ubuntu EC2 instance.

The public subnet must be associated with a route table whose default route targets the Internet Gateway. The EC2 instance also needs a public IPv4 address and appropriate network access to be reachable from the internet. For safer operation, SSH should be restricted to the administrator’s IP address rather than exposed to all addresses.

5. Implementation Procedure

5.1 AWS infrastructure: Create a custom VPC with CIDR 10.0.0.0/16, create a public subnet with CIDR 10.0.1.0/24, attach an Internet Gateway, configure a route table with 0.0.0.0/0 routed to the Internet Gateway, associate the route table with the subnet, and create a security group for SSH and HTTP. Launch an Ubuntu EC2 instance in the subnet and connect using SSH.

5.2 Ubuntu configuration: Refresh package metadata and apply available updates. Install Git, Apache, Curl, and Vim using the Ubuntu package manager. Verify installations and check that Apache is active.

5.3 Website deployment: Create index.html with the project title, student name, batch code, current date, and the message “Hosted on AWS Ubuntu Server”. Place the page in Apache’s document root (commonly /var/www/html/index.html), confirm Apache is running, and open http://<EC2-Public-IP>/ in a browser.

5.4 Git and GitHub: Initialize the local repository, stage the project files, commit them, create a feature branch, commit a change on that branch, switch to main, merge the feature branch, configure the GitHub remote, and push main. Verify the files and commit history on GitHub.

6. Commands Reference

Update packages: `sudo apt update && sudo apt upgrade -y`

Install required tools: `sudo apt install -y git apache2 curl vim`

Check Apache status: `sudo systemctl status apache2`

Enable and start Apache: `sudo systemctl enable --now apache2`

Check installed packages: `dpkg -l git apache2 curl vim`

Check repository status: `git status`

Stage and commit: `git add . && git commit -m "Add project files"`

Create a branch: `git switch -c feature-documentation`

Return and merge: `git switch main && git merge feature-documentation`

Configure remote and push: `git remote add origin <repository-url> && git push -u origin main`

These are representative commands for the documented procedure. The report does not assert that every command was executed successfully unless confirmed by project evidence.

7. Bash Automation Plan

7.1 Server health check (health-check.sh): The assignment calls for reporting CPU usage, memory usage, disk usage, logged-in users, running processes, and server uptime. Typical Linux utilities include top or vmstat, free, df, who, ps, and uptime.

7.2 Apache service monitor (apache-monitor.sh): The script should test the Apache service state, restart Apache if it is inactive, and print a clear success or failure message. On Ubuntu, the service name is normally apache2; scripts should check command exit status and run with the permissions required to manage the service.

7.3 Website backup (backup.sh): The script should create a timestamped compressed archive of the website files and save it in a dedicated backup directory. The backup path should be writable by the script, and old backups may need a retention policy.

These descriptions follow the assignment brief. The repository README lists the three scripts, but the individual script files were not retrievable at the expected root paths during this review. Their contents and runtime behaviour therefore remain to be verified.

8. Log Monitoring and Troubleshooting

The assignment requires a log report named log-report.txt covering failed login attempts, SSH login activity, system errors, and warning messages. On Ubuntu, relevant sources may include the systemd journal and

authentication logs. Example inspection commands include sudo journalctl -p warning..alert, sudo journalctl -u ssh, and sudo grep -i "failed\|invalid" /var/log/auth.log where that file is available.

Log access and file locations vary by Ubuntu version and logging configuration. The final report should contain actual findings from the target instance, with sensitive information removed. Do not fabricate log events or include passwords, tokens, private keys, or other secrets.

9. Repository Structure

Expected project layout based on the assignment brief:

```
DevOps-Fundamentals-Project/
■■■ README.md
■■■ index.html
■■■ health-check.sh
■■■ apache-monitor.sh
■■■ backup.sh
■■■ log-report.txt
■■■ screenshots/
■   ■■■ aws-setup.png
■   ■■■ website.png
■   ■■■ github.png
■■■ documentation/
```

■■■ Project_Report.pdf

The repository README also describes the AWS architecture, installation steps, commands, folder structure, challenges, and learning outcomes. Actual screenshots and script files should be committed to the repository before final submission.

10. Challenges and Troubleshooting

The repository README records troubleshooting Git authentication and repository configuration issues. These are common when configuring a remote repository or authenticating pushes. A second documented focus is verifying the website by checking Apache’s service status and testing the EC2 public IP address.

Useful diagnostic checks include git remote -v, git status, git log --oneline --graph --all, sudo systemctl status apache2, sudo journalctl -u apache2, and checking that the EC2 security group and subnet route allow HTTP traffic. If GitHub rejects a push, verify the remote URL and authenticate using Git Credential Manager, a valid token, or SSH; never place credentials in repository files.

11. Testing and Verification

• AWS: Confirm the VPC, subnet, Internet Gateway, route table association, security group, and EC2 instance configuration in the AWS console.

• Linux: Confirm package installation and that Apache is active using systemctl.

• Website: Open the EC2 public IPv4 address in a browser and capture a screenshot showing the deployed page.

• Git: Confirm the initial commit, feature branch, merge, and push using git log and the GitHub repository page.

• Automation: Run each Bash script on Ubuntu and save evidence of the output and recovery/backup behaviour.

• Logs: Generate log-report.txt from the actual instance logs and review it for secrets before committing.

Evidence checklist: the repository link was supplied by the project owner. During this review, the README.md was accessible through the GitHub connector; the expected root paths index.html, health-check.sh,

apache-monitor.sh, backup.sh, and log-report.txt were not retrievable at those exact paths. This may mean they are absent, stored elsewhere, or named differently. Verify the current repository contents before treating these deliverables as complete.

12. Learning Outcomes

The project is designed to build practical understanding of Ubuntu server administration, package installation, Apache hosting, AWS networking, EC2 access, Git workflows, basic Bash automation, log inspection, troubleshooting, and technical reporting. The repository README specifically records improved Bash scripting, documentation, and problem-solving skills as learning outcomes.

13. Conclusion

This project brings together the basic steps involved in hosting a static website on a cloud-based Linux server and managing project files with GitHub. It provides a foundation for further DevOps practice in automation, operational monitoring, repeatable deployments, and secure cloud administration. Final evaluation should be supported by screenshots, verified script execution, and the generated log report.

References

1. Project assignment: “ITVedant DevOps Fundamentals Mini Project” (uploaded PDF).

2. Project repository: https://github.com/VaibhavR12-Gh/DevOps-Fundamentals-Project

3. AWS, Ubuntu, Apache, Git and GitHub documentation may be consulted for environment-specific commands and troubleshooting.